

Some sort of logo

DVWA Penetration Test

Reflected Cross-Site Scripting (XSS) Assessment

by Ryyni-CK

Target: <http://193.167.189.112:2167/>

Start of testing: July 27, 2026

End of testing: July 27, 2026

1. Executive Summary

A security assessment was performed against the Damn Vulnerable Web Application (DVWA) instance hosted at <http://193.167.189.112:2167/>. The engagement focused on the application's input-handling behaviour, in particular the **XSS (Reflected)** module. Testing confirmed a reflected Cross-Site Scripting vulnerability that allows arbitrary JavaScript to execute in the victim's browser in the context of the application.

The module implements a randomised character-substitution filter (`strtr()`) rather than proper output encoding. Because this substitution is a fixed, reversible one-to-one mapping, it does not prevent the attack; it only rearranges the special characters. By recovering the mapping and supplying the inverse, a fully working payload was reconstructed and executed. One finding of **Medium** severity is reported, together with concrete remediation guidance.

Finding	Severity	Status
Reflected XSS via reversible substitution filter	Medium	Confirmed / Exploited

2. Scope and Objectives

The assessment was limited to the following target and objective. Only the systems listed below were in scope; no other hosts, networks, or applications were tested.

Item	Detail
Target application	Damn Vulnerable Web Application (DVWA) v1.9
Target URL	http://193.167.189.112:2167/
Module under test	XSS (Reflected)
Security level	low (randomised implementation)
Test type	Authenticated, grey-box (source code available via "View Source")
Objective	Determine whether user input is reflected without adequate encoding and can trigger script execution via <code>alert()</code> .

3. Methodology

Testing followed a standard web-application testing approach aligned with the OWASP Testing Guide (input-based vulnerabilities). The workflow was:

- **Reconnaissance / source review.** The module's source was read via the "View Source" button to understand how the name parameter is processed server-side.
- **Input probing.** Individual special characters were submitted to observe how the application transformed them in the reflected response.
- **Filter analysis.** The transformation was identified as a fixed substitution cipher and its mapping was recovered.

- **Exploitation.** The mapping was inverted to build an input string that the server reassembles into a valid XSS payload.
- **Verification.** The payload was submitted and confirmed to execute (`alert()` dialog observed).

4. Detailed Finding – Reflected Cross-Site Scripting

Attribute	Value
Vulnerability	Reflected Cross-Site Scripting (XSS)
Location	XSS (Reflected) module, name GET parameter
Severity	Medium
CVSS v3.1	6.1 – AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
CWE	CWE-79: Improper Neutralization of Input During Web Page Generation
OWASP	A03:2021 – Injection

4.1 Description

The application reflects the value of the name parameter directly into the HTML response. Instead of HTML-encoding the value, the code passes it through a three-argument `strtr()` call that substitutes each special character for another special character, according to a list shuffled with a fixed seed. The relevant server-side logic is reproduced below:

```
$tr_chars = "\"'\/<>()[!^~;:~+";
$rnd_mapping = str_split($tr_chars);
shuffle($rnd_mapping); // seeded with a fixed srand() value
$str_random = implode("", $rnd_mapping);

$namestr = strtr($_GET['name'], $tr_chars, $str_random);
echo '<pre>Hello ' . $namestr . '</pre>';
```

Two properties make this ineffective as a security control. First, the substitution is a **one-to-one mapping** over the special-character set – no character is removed or encoded, only swapped. Second, the seed is fixed, so the mapping is **constant** for the instance and can be recovered by observation. The set is closed, so every character required for a payload (`< > ( ) ' " /`) can be produced by submitting its pre-image.

4.2 Steps to Reproduce

Step 1 – Recover the mapping. Submit the full special-character set as the name value:

```
"'\/<>()[!^~;:~+</pre>
```

The application returned the following reflected string, which lines up with the input character-by-character:

```
Hello ;>\}{`['<^}+/$":?~(
```

Step 2 – Invert the characters needed for the payload. Reading the two rows column-by-column gives, for each character required in the output, the character that must be typed to produce it:

Desired output	Character to type
< (open tag)	]

> (close tag)	'
((open paren)	\$
) (close paren)	)
' (single quote)	[
/ (slash)	` (backtick)
" (double quote)	;

Characters that are not part of the set – letters, digits, spaces, = and . – pass through unchanged and require no substitution.

4.3 Proof of Concept

Step 3 – Build and submit the payload. A compact payload that requires only < > () and the quotes is an <svg> element with an onLoad handler. Applying the inverse mapping above, the exact string entered into the input box is:

```
]svg onload=alert$[XSS()]'
```

The server passes this through `strtr()`, which reassembles it into the following markup that is written into the page and executed by the browser:

```
<svg onload=alert('XSS')>
```

On submission the browser rendered the injected element and executed the handler, displaying an alert dialog containing the text **XSS**. This confirms arbitrary JavaScript execution. An equivalent classic payload, `<script>alert('XSS')</script>`, was produced from the input below (the backtick becomes the / of the closing tag):

```
]script'alert$[XSS[]]`script'
```

4.4 Impact

Reflected XSS allows an attacker to execute JavaScript in a victim's browser in the security context of the vulnerable origin, typically by luring the victim into clicking a crafted link. Realistic consequences include:

- Theft of session cookies or tokens, leading to session hijacking and account takeover.
- Performing actions as the victim (e.g. changing settings, submitting forms) without consent.
- Rendering fake login prompts or content to harvest credentials (phishing within a trusted site).
- Keylogging, browser-based pivoting, and delivery of further client-side attacks.

Because the substitution filter gives a false impression of protection, the issue may go unnoticed while remaining fully exploitable, which increases its practical risk.

4.5 Remediation

The root cause is that untrusted input is written into HTML without contextual output encoding. Recommended fixes, in order of importance:

- **Encode on output.** Apply context-aware encoding when reflecting user input. In PHP, wrap the value with `htmlspecialchars($input, ENT_QUOTES, 'UTF-8')` so characters such as `<` become `<`; and are rendered as text, never as markup. This is exactly what the higher DVWA levels demonstrate.
- **Do not rely on character substitution or blacklists.** Reversible or incomplete filters (such as the `strtr()` mapping here) do not stop injection.
- **Validate input.** Enforce expected type, length, and character set on the server side as defence in depth, in addition to (not instead of) output encoding.
- **Deploy a Content-Security-Policy.** A strict CSP that disallows inline script significantly reduces the impact of any XSS that slips through. Note the application currently sends `X-XSS-Protection: 0`, disabling the legacy browser filter.
- **Set HttpOnly and Secure cookie flags.** `HttpOnly` prevents script from reading session cookies, limiting the value of a successful XSS.

5. Conclusion

The DVWA XSS (Reflected) module is vulnerable to reflected Cross-Site Scripting. The randomised `strtr()` substitution is not an effective control: it is a fixed, reversible mapping that was recovered by observation and inverted to reconstruct a working payload (`<svg onload=alert('XSS')>`), which executed successfully. The vulnerability should be remediated by applying proper output encoding (`htmlspecialchars()`) and supported by a strict Content-Security-Policy. The finding is rated Medium severity.

Appendix A – Character Mapping Reference

Full recovered mapping for this instance (input character → reflected output character). The mapping is specific to the fixed seed of the tested instance and would differ for a different seed.

In	Out	In	Out	In	Out
"	;	)	)	^	!
'	>	[	'	~	\$
\	\	]	<	;	"
/	]	{	^	:	:
<	{	}	}	?	?
>	`	!	+	+	~
(	[	`	/	\$	(

This report was produced for an authorised security training exercise against an intentionally vulnerable application (DVWA).

Confidential.